Software Development

Chapter 1: Highway to Shell

Benjamin Hackl

Department of Mathematics and Scientific Computing @ University of Graz

Welcome to working with shells!



- Textual interface to your operating system
- To interact with a *shell prompt*, you need a *terminal*
- There are many different shells, we will focus on the *Bourne-Again SHell* `bash`
  - other (popular) shells: `zsh`, `csh/tcsh`, `fish`, `ksh`, `cmd.exe`, `PowerShell`
  - https://en.wikipedia.org/wiki/Comparison_of_command_shells

Choose your Bash environment

- **Browser:** WebVM – Bash in your browser, no installation needed

  (great to follow along + exercises)

- **Linux / macOS:** open a terminal, run `bash`
- **Windows:** WSL or Git Bash

Type the command after `$`, not the prompt itself.

Using the shell

Launch a new terminal, you will find a *prompt* like

```
somepc:~$
```

- `somepc` is the machine name, the *current working directory* is `~` (short for your "home"), `$` conventionally indicates a non-admin user (`#` often indicates root).

Shells allow interaction with the OS by *executing programs*

```
somepc:~$ date
Mi 02 Okt 2024 12:10:19 CEST
somepc:~$ echo hello world
hello world
somepc:~$
```

How does the shell know programs?

- Commands can be aliases, functions, or shell builtins (e.g. `cd`, `echo`).
- For external programs without a `/` in their name, Bash searches `$PATH`: a colon-separated list of directories.
- Use `type` to inspect command resolution; `which` often only searches `$PATH`.

```
somepc:~$ echo "$PATH"
/home/behackl/.local/bin:/usr/local/sbin:/usr/local/bin:/usr/bin
somepc:~$ type echo
echo is a shell builtin
somepc:~$ /usr/bin/echo direct access
direct access
```

File system: navigating the `/path/to/success`

- Paths are (separated) lists of directories; Linux + macOS: `/`, Windows: `\` as separator
- **Filesystem root:** `/` for Linux + macOS; Windows drive paths start at e.g. `C:\`
- Useful abbreviations: `.` is current, `..` is parent directory

```
somepc:~$ pwd  ## pwd ... print working directory
/home/behackl
somepc:~$ cd /home  ## cd ... change directory
somepc:/home$ pwd
/home
somepc:/home$ cd ..
somepc:/$ ls  ## ls ... list directory contents
bin                 etc    lost+found  root             srv  var
boot                home   mnt         rootfs-pkgs.txt  sys
desktopfs-pkgs.txt  lib    opt         run              tmp
dev                 lib64  proc        sbin             usr
somepc:/$ cd  ## cd without arguments takes you $HOME
somepc:~$
```

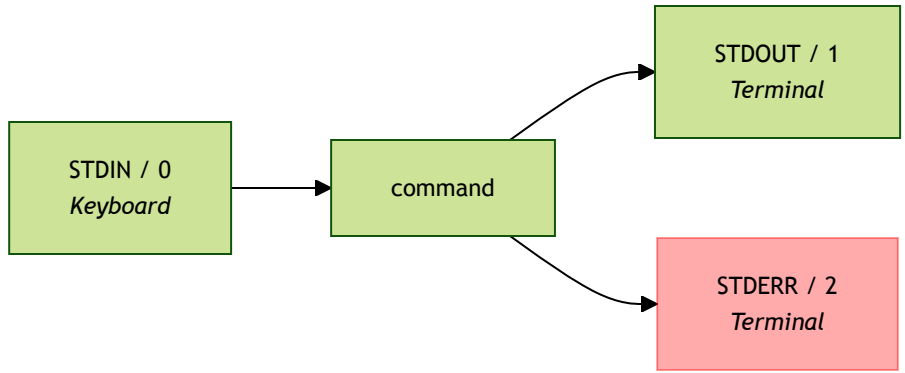
Flags, options, and the `man`ual

- Behavior of programs can be modified by passing *flags and options* as arguments
- `-h` or `--help` often print usage information, but not universally: `ls -h` means human-readable sizes; `ls --help` below requires GNU `ls`.
- Bash builtins have their own help, e.g. `help cd`.

```
somepc:~$ ls --help
Usage: ls [OPTION]... [FILE]...
List information about the FILEs (the current directory by default).
...
somepc:~$ ls -la  ## same as ls -l -a
drwx------  2 behackl behackl    4096  2. Okt 14:07 .
drwxr-xr-x  4 root    root       4096 26. Sep 13:23 ..
-rw-r--r--  1 behackl behackl    16    2. Okt 14:12 .secret-uncovered
```

- Manuals: `man ls`, `info ls`; search: `apropos signal`; exit with `q`.

IO Streams: Input, Output, Errors



- Streams can be *rerouted*, e.g. to files: `< file`, `> file`, `2> file`
- `2>&1` sends STDERR to STDOUT's current destination; `>> file` appends STDOUT.

```
somepc:~$ echo hello world > file.txt
somepc:~$ cat file.txt  ## cat ... con_cat_enate files, print to STDOUT
hello world
somepc:~$ cat < file.txt > file2.txt
somepc:~$ cat file2.txt
hello world
```

Chaining programs: piping

- Pipe `cmd1 | cmd2` reroutes **STDOUT** of `cmd1` to STDIN of `cmd2`, not STDERR.

```
somepc:~$ ls -l / | tail -n1  ## show only last line of output
drwxr-xr-x  12 root root  4096  6. Sep 19:19 var
somepc:~$ curl --silent --head -L uni-graz.at
HTTP/1.1 301 Moved Permanently
...  (lots of output)
somepc:~$ curl --silent --head -L uni-graz.at | grep --ignore-case '^location:' | cut -f2 -d ' '
https://uni-graz.at/
https://www.uni-graz.at/
https://www.uni-graz.at/de/
```

- `curl` – send web requests
- `grep` – print lines matching a pattern (Get Regular ExPression; name from the editor command `g/re/p`)
- `cut` – select sections from lines

Bash History

- `history` lists the in-memory history; `history 10` shows the last 10 entries. Bash normally saves it to `~/.bash_history` on exit (configurable).

```
somepc:~$ !!  ## execute last command
somepc:~$ !n  ## execute history entry number n
somepc:~$ !-n  ## execute nth command back
somepc:~$ !string  ## last command starting with string
somepc:~$ !?string?  ## last command containing string
```

Bash Shortcuts

Command	*Description*
`Tab`	Autocomplete command/argument
`Tab` `Tab`	Show possible completions
`Ctrl` + `c`	Send `SIGINT` to running command
`Ctrl` + `l`	Clear the terminal

Command	*Description*
`Ctrl` + `a`	Jump to beginning of line
`Ctrl` + `e`	Jump to end of line
`Ctrl` + `u`	Delete line left of cursor
`Ctrl` + `w`	Delete word left of cursor

Job control / 1

- By default, shell blocks while executing commands
- Queue multiple commands in sequence with `;`

```
somepc:~$ sleep 10; echo done!
done!
```

- Jobs can be started *in the background* by appending `&`

```
somepc:~$ sleep 1000 &
somepc:~$
```

- Foreground jobs can be *paused* via `Ctrl` + `z`
- `bg` resumes a stopped job in the background; `jobs` lists shell jobs (including stopped ones); `fg %1` brings job 1 to the foreground.

Job control / 2

> ⚠ **Warning**
>
> Background jobs **may** receive `SIGHUP` when the terminal/session closes. `disown` stops Bash managing the job, but does not detach its terminal I/O. For work that must survive disconnects, prefer `tmux` / `screen`, or `nohup command > run.log 2>&1 &`.

- `ps` shows a selection of processes
- `top` / `htop` / `btop` show live process dashboards (quit with `q`)
- `kill` sends signals to processes (PID) or shell jobs (e.g. `%1`)
  - Try `kill 420` first: `SIGTERM` allows cleanup.
  - Last resort: `kill -KILL 420` (`kill -9 420`); `SIGKILL` cannot be caught.
  - List signals: `kill -l`; Linux details: `man 7 signal`.

File Modes / Permissions

- Files and friends have *permission bits*, executing a file directly requires `x` bit

```
somepc:~$ ls -l
-rwxr-xr-x 1 behackl behackl 151168 Oct  3 09:34 cp
-rw-r--r-- 1 behackl behackl     17 Oct  3 09:34 file.txt
drwxr-xr-x 2 behackl behackl   4096 Oct  3 09:34 subdir
somepc:~$ ./file.txt
-bash: ./file.txt: Permission denied
```

- `d` ... directory, `r` ... read, `w` ... `write`, `x` ... execute/search (for dirs)
- Order: *file type* / 3 *owner* bits / 3 *group* bits / 3 *other* bits
- `chmod` changes permissions:
  - `chmod o-x cp` removes `x`-bit for *other* from `cp`,
  - `chmod ug+w file.txt` makes `file.txt` user-/group-writeable,
  - `chmod u+x file.txt` adds execute permission for the owner (`chmod +x` is affected by your `umask`)
- Numeric mode: $(r, w, x) \triangleq (4, 2, 1)$; `chmod 744 file.txt` sets `rwxr--r--`

How does Linux execute files?

- Native binaries (compiled from C, C++, Rust, …) use a format the kernel just knows™ how to deal with
- For executable scripts, a *shebang* `#!` names the interpreter the kernel runs
  - `#!/bin/bash` selects Bash at that path
  - `#!/usr/bin/env python3` selects `python3` using the caller's `$PATH`
- Without a shebang, the kernel does **not** automatically choose Bash; Bash may fall back to interpreting the file after an executable-format error
- Explicit `bash script.sh` or `python3 script.py` selects that interpreter, regardless of the shebang

References

If this was too fast, have a look at the following:

- MIT Course *The Missing Semester*
  - https://missing.csail.mit.edu/2020/course-shell/
  - https://missing.csail.mit.edu/2020/shell-tools/
  - https://missing.csail.mit.edu/2020/command-line/
- https://cs.lmu.edu/~ray/notes/bash/
- https://learn.microsoft.com/en-us/training/modules/bash-introduction/